ROME Demo Assignment Report

Huancheng Yang hy3281

Part III: Debugging Loop and Prompt Iterations (Adder)

This section documents my iterative improvement process for the Part II adder design, focusing on the hierarchical generation of `full_adder` and the downstream `adder4/adder8` builds. The main failure mode was *duplicate module definitions* caused by the LLM inlining submodule code into a file that should contain only one module. The notebook records each attempt via `log_iter_*.txt`, and I used the simulator/compiler output to guide fixes. The process ends with a final passing run as required.
(Requirement: at least three iterations, include key error/output lines, end with a passing run.)

Iteration 1: Duplicate module declarations (compile error)

What failed: When compiling `full_adder` (and later `adder8`), `iverilog` reported that `half_adder` (and other modules) had already been declared in the same scope. This happened because the generated `full_adder.v` (or `adder8.v`) incorrectly *re-defined* submodules instead of only instantiating them.

Key compiler output snippet:

```
half_adder/half_adder.v:1: error: 'half_adder' has already been declared in this scope.
full_adder/full_adder.v:1: : It was declared here as a module.
half_adder/half_adder.v:4: Module half_adder was already declared here: full_adder/full_adder.v:1
```

Change applied: I strengthened the prompt to explicitly say "do not create any supplementary modules" and "only output Verilog code, no explanations". This matched the intended behavior of the provided feedback loop but was still not strict enough for hierarchical generation.

Outcome: The issue persisted in later generations (the model still sometimes inlined submodule definitions).

Iteration 2: Enforce file-level contract (still intermittent)

What failed: Even with clearer instructions, the model occasionally produced a file containing multiple `module ... endmodule` blocks (e.g., `adder8.v` containing `half_adder`, `full_adder`, `adder4`, etc.). This again caused duplicate declarations when compiling all files together.

Key compiler output snippet (example):

```
adder4/adder4.v:1: error: 'half_adder' has already been declared in this scope.
adder8/adder8.v:1: : It was declared here as a module.
...
```

Change applied: I introduced a *strict output contract* in the prompt:

- Output must contain **exactly one** module definition total.
- The first non-empty line must be `module <TARGET>(`.
- Forbid any other module names in the output using explicit forbidden substrings.

Outcome: The generation quality improved significantly and became consistent with hierarchical instantiation.

Iteration 3: Final strict prompts + correct hierarchy instantiation (pass)

What failed previously: The top-level behavior needed to be strictly hierarchical: `full_adder` must *instantiate* `half_adder` twice; `adder4` must instantiate `full_adder` four times; `adder8` must instantiate `adder4` twice.

Final change applied: I used strict prompts that:

- For `full_adder`, explicitly require two `half_adder` instances (`ha1`, `ha2`) and `cout = c1 | c2`.
- For `adder8`, explicitly require two `adder4` instances (lower 4 bits / upper 4 bits) to satisfy the assignment requirement that the top module instantiates at least two previously generated submodules.

Outcome: All testbenches passed. The final `iverilog/vvp` outputs ended with `passed!` for each module, and the full hierarchy compiled without duplicate-module errors.

Iteration Summary Table

Iter.	Issue Observed	Fix Applied
1	Duplicate module declarations (e.g., `half_adder` declared twice)	Strengthen prompt: no supplement modules; code-only output
2	Multi-module output in a single `.v` file (inlined submodules)	Add strict file-level contract: exactly module, forbid other module names
3	Ensure true hierarchical instantiation (FA uses 2 HA; adder8 uses 2 adder4)	Final strict prompts with explicit inst tiation requirements for each module

Report

1 Model and Generation Settings

I used the provided ROME notebook workflow in Google Colab with `iverilog` and `vvp`. For the LLM wrapper, I used the ChatGPT option with model choice `gpt-4o-mini` (as set in the notebook). I kept the default feedback-loop structure and used the hierarchical loop (`hier_gen`) to generate submodules in order.

2 Part I: Provided Mux Hierarchy Demo

I ran the provided mux hierarchy demo and generated `mux2to1`, `mux4to1`, and `mux8to1`. After adding/using self-checking testbenches, each module compiled and simulated successfully with final output line containing `passed!`. This matches the requirement that the notebook reaches "Testbench ran successfully" and the testbench prints `passed!`.

3 Part II: Extension Design — Ripple-Carry Adder Hierarchy

I implemented Option A (recommended): a ripple-carry adder hierarchy:

- `half_adder`: 1-bit half adder
- `full_adder`: 1-bit full adder built from two half adders
- `adder4`: 4-bit ripple-carry adder built from four full adders
- `adder8`: 8-bit top module built from two `adder4` blocks

3.1 Hierarchical Decomposition and Instantiation

Module	Interface (summary)	Instantiates
`half_adder`	`a,b` → `sum,carry`	none
`full_adder`	`a,b,cin` → `sum,cout`	2× `half_adder`
`adder4`	`a[3:0],b[3:0],cin` → `sum[3:0],cout`	4× `full_adder`
`adder8` (top)	`a[7:0],b[7:0],cin` → `sum[7:0],cout`	2× `adder4`

This satisfies the assignment requirement that the top module instantiates at least two previously generated submodules.

3.2 Testbenches

Each module has a self-checking testbench that prints a clear final line containing `passed!` on success, consistent with the notebook parsing requirement.

4 Final Prompts (Top + Representative Submodule)

As required, I include my final prompt for the top module (`adder8`) and one representative submodule (`full_adder`). The key idea is to force *single-module output per file* and hierarchical instantiation only.

Final Prompt: adder8 (Top)

```
CRITICAL OUTPUT CONTRACT:
- Output Verilog code only.
- EXACTLY ONE module definition total.
- FIRST non-empty line: module adder8(
- Forbid defining any other module name in this file.

Module name: adder8
Ports (exact):
input wire [7:0] a
input wire [7:0] b
input wire cin
output wire [7:0] sum
output wire cout

Hierarchy requirement (HARD):
Instantiate TWO adder4 blocks:
- low4: a[3:0], b[3:0], cin -> sum[3:0], c4
- high4: a[7:4], b[7:4], c4 -> sum[7:4], cout
Do NOT define adder4/full_adder/half_adder in this file.
```

Final Prompt: full_adder (Representative Submodule)

```
CRITICAL OUTPUT CONTRACT:
- Output Verilog code only.
- EXACTLY ONE module definition total.
- FIRST non-empty line: module full_adder(
- Forbid defining any other module name in this file.

Module name: full_adder
Ports (exact):
input wire a
```

```
input wire b
input wire cin
output wire sum
output wire cout

Hierarchy requirement (HARD):
- Instantiate TWO half_adders (existing module half_adder):
  ha1: a,b -> s1,c1
  ha2: s1,cin -> sum,c2
- cout = c1 | c2
Do NOT define module half_adder in this file.
```

5 Final Verification Commands and Passing Confirmation

I verified each module using `iverilog` (Verilog-2012 mode) and `vvp`. The final commands used:

Half Adder

```
iverilog -g2012 -o sim_ha.out half_adder/half_adder.v half_adder/half_adder_tb.v
vvp sim_ha.out
```

Full Adder

```
iverilog -g2012 -o sim_fa.out full_adder/full_adder.v full_adder/full_adder_tb.v half_adder/
    half_adder.v
vvp sim_fa.out
```

Adder4

```
iverilog -g2012 -o sim_a4.out adder4/adder4.v adder4/adder4_tb.v full_adder/full_adder.v half_adder
    /half_adder.v
vvp sim_a4.out
```

Adder8 (Top)

```
iverilog -g2012 -o sim_a8.out adder8/adder8.v adder8/adder8_tb.v adder4/adder4.v full_adder/
    full_adder.v half_adder/half_adder.v
vvp sim_a8.out
```

In each case, the simulation ended with the testbench printing `passed!`, confirming correctness for Part I and Part II designs.

```
%%bash
set -e
cd /content

echo "=== half_adder ==="
iverilog -g2012 -o sim_ha.out half_adder/half_adder.v half_adder/half_adder_tb.v
vvp sim_ha.out

echo "=== full_adder ==="
iverilog -g2012 -o sim_fa.out full_adder/full_adder.v full_adder/full_adder_tb.v half_adder/half_adder.v
vvp sim_fa.out

echo "=== adder4 ==="
iverilog -g2012 -o sim_a4.out adder4/adder4.v adder4/adder4_tb.v full_adder/full_adder.v half_adder/half_adder.v
vvp sim_a4.out

echo "=== adder8 ==="
iverilog -g2012 -o sim_a8.out adder8/adder8.v adder8/adder8_tb.v adder4/adder4.v full_adder/full_adder.v half_adder/half_adder.v
vvp sim_a8.out
```

```
=== half_adder ===
passed!
=== full_adder ===
passed!
=== adder4 ===
passed!
=== adder8 ===
passed!
```